

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №0
«Решение уравнения 6 степени»

Выполнил:
Пахомов Владимир Юрьевич, студент группы N3250

(подпись)

Проверил:
Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург
2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АЛГОРИТМ РЕШЕНИЯ УРАВНЕНИЯ.....	4
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ.....	7
3. БЛОК СХЕМА АЛГОРИТМА.....	10
4. ТЕСТИРОВАНИЕ АЛГОРИТМА.....	11
5. ИСХОДНЫЙ КОД АЛГОРИТМА.....	14
ЗАКЛЮЧЕНИЕ.....	21
ПРИЛОЖЕНИЕ А.....	22

ВВЕДЕНИЕ

Цель работы — разработать программу для нахождения корней уравнения 6 степени вида:

$$ax^6 + bx^5 + cx^4 + dx^3 + nx^2 + mx + k = 0, \quad a, b, c, d, n, m, k \in \mathbb{R}$$

Для нахождения корней нужно использовать следующий алгоритм:

- при $a \neq 0$ найти первый корень методом касательных;
- при $a = 0, b \neq 0$ найти первый корень методом хорд;
- при $a = 0, b = 0, c \neq 0$ найти первый корень комбинированным методом хорд и касательных;
- при $a = 0, b = 0, c = 0, d \neq 0$ найти три корня методом Кардано;
- при $a = 0, b = 0, c = 0, d = 0, n \neq 0$ найти два корня методом Виета;
- при $a = 0, b = 0, c = 0, d = 0, n = 0, m \neq 0$ найти корень, равный $-k / m$;
- при $a = 0, b = 0, c = 0, d = 0, n = 0, m = 0, k \neq 0$ вывести, что решений нет;
- при $a = 0, b = 0, c = 0, d = 0, n = 0, m = 0, k = 0$ вывести, что x является любым числом.

После нахождения каждого корня степень уравнения понижается при помощи схемы Горнера. Получившееся после понижения степени уравнение решается по такому же алгоритму.

1. АЛГОРИТМ РЕШЕНИЯ УРАВНЕНИЯ

$$ax^6 + bx^5 + cx^4 + dx^3 + nx^2 + mx + k = 0, \quad a, b, c, d, n, m, k \in \mathbb{R}$$

1) Проверка коэффициентов:

- если $a \neq 0$, то уравнение степени 6. Ищем интервал с корнем;
- если $a = 0, b \neq 0$, то уравнение степени 5. Ищем интервал с корнем;
- если $a = 0, b = 0, c \neq 0$, то уравнение степени 4. Ищем интервал с корнем;
- если $a = 0, b = 0, c = 0, d \neq 0$, то уравнение степени 3. Используем метод Кардано;
- если $a = 0, b = 0, c = 0, d = 0, n \neq 0$, уравнение степени 2. Используем теорему Виета.

2) Поиск интервала:

Для начала нужно определить границы, в которых будет искаться корень. Для этого используем теорему Коши, которая гласит, что корни находятся внутри промежутка $[-R; R]$, где R :

$$R = 1 + \frac{\max(|a_{n-1}| \dots |a_0|)}{a_n}$$

Далее нужно разбить этот диапазон на мелкие отрезки, например на $\frac{R}{10000}$, и потом идём по ним, пока не найдём две соседние точки, в которых значения функции $f(x)$ различаются (условие Больцано-Коши).

3) Метод касательных

Для корня полинома 6 степени используется метод касательных. Это итерационный метод, где следующее приближение x_{n+1} находится как точка пересечения касательной к графику функции в точке x_n с осью абсцисс.

$$\text{Формула: } x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

После завершения метода переходим к пункту 4.

4) Метод хорд

Для нахождения полинома 5 степени используется метод хорд. Вместо касательной используется хорда, проведённая через концы интервала $[l, r]$, на котором функция меняет знак. Корень ищется как точка пересечения этой хорды с осью ОХ.

$$\text{Формула: } x = l - \frac{f(l) \cdot (r - l)}{f(r) - f(l)}$$

Если значение $|f(r) - f(l)|$ близко к нулю или , то сдвигаем промежуток l на eps вправо.

Если значение функции $f(x)$ меньше заданной точности eps , то возвращаем корень. Если на данном шаге требуемая точность ещё не достигнута, то сужаем интервал по следующему условию:

- если $f(l) \cdot f(r) < 0$, то $r = x$
- иначе $l = x$

После завершения метода переходим к пункту 5.

5) Комбинированный метод (метод хорд и касательных)

Для нахождения корня полинома степени 4 используется комбинированный метод. Этот метод приближается к корню сразу с двух сторон. С одной стороны применяется метод касательных, с другой – метод хорд. Для определения с какой стороны использовать хорду, а с какой касательную, используется условие $f(x) \cdot f''(x) > 0$.

После выполнения метода переходим к пункту 6.

6) Метод Кардано

Для нахождения корней полинома степени 3 используется метод Кардано. Для начала уравнение приводится к каноническому виду $y^3 + py + q = 0$, затем вычисляется дискриминант $Q = \left(\frac{p}{3}\right)^3 + \left(\frac{q}{2}\right)^2$. Далее проверяем условия:

– если $Q > 0$, то находим единственный вещественный корень

$$y = \sqrt[3]{\frac{-q}{2} + \sqrt{Q}} + \sqrt[3]{\frac{-q}{2} - \sqrt{Q}};$$

– если $Q = 0$, то находим три вещественных корня, два из которых будут кратными

$$y_1 = 2 \cdot \sqrt[3]{\frac{-q}{2}}, \quad y_{2,3} = -\sqrt[3]{\frac{-q}{2}};$$

– если $Q < 0$, то уравнение имеет три различных вещественных корня. Для этого сначала нужно сделать геометрическую подстановку:

$$\varphi = \arccos\left(-\frac{q}{2 \cdot \sqrt{-\frac{p^3}{27}}}\right), \quad t = 2 \cdot \sqrt{-\frac{p}{3}}$$

$$y_1 = t \cdot \cos \frac{\varphi}{3}, \quad y_2 = t \cdot \cos \left(\frac{\varphi}{3} + \frac{2\pi}{3}\right), \quad y_3 = t \cdot \cos \left(\frac{\varphi}{3} + \frac{4\pi}{3}\right)$$

После нахождения корней делается обратная замена $x_i = y_i - \frac{a}{3}$.

7) Теорема Виета

Для нахождения двух корней полинома степени 2 используется теорема Виета. Для нахождения корней сначала находим дискриминант $D = b^2 - 4ac$. Далее

- если $D > 0$, то возвращаем два корня $x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}$;
- если $D = 0$, то возвращаем один корень $x = \frac{-b}{2a}$;
- если $D < 0$, то корней нет.

8) Схема Горнера

Схема Горнера используется для понижения степени полинома после нахождения корня. Полином $a_n x^n + \dots + a_0$ делится на $(x - root)$ и коэффициенты для нового полинома $b_{n-1}x^{n-1} + \dots + b_0$ пересчитываются по формулам:

$$b_{n-1} = a_n$$

$$b_i = b_{i+1} \cdot root + a_{i+1}$$

2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Таблица 1— Используемые переменные

Название переменной	Тип переменной	Границы типа	Назначение переменной
solutions	number[]	-	Хранит все найденные корни
coeffs	number[]	-	Хранит коэффициенты уравнения
eps	number	-1.79769e+308 до 1.79769+308	Точность для поиска корней
N	number	0 до 9007199254740991	Количество итераций
x0	number	-1.79769e+308 до 1.79769+308	Начальное приближение для метода касательных
l	number	-1.79769e+308 до 1.79769+308	Левая граница поиска корня для метода хорд
r	number	-1.79769e+308 до 1.79769+308	Правая граница поиска корня для метода хорд
root	number	-1.79769e+308 до 1.79769+308	Хранит текущий найденный корень
res	number	-1.79769e+308 до 1.79769+308	Используется для расчёта значения функции в точке
i	number	0 до 9007199254740991	Используется для итераций в циклах
cleanedCoeffs	number[]	-	Хранит ненулевые коэффициенты в функции поиска границ корней
zeroFlag	boolean	true / false	Флаг, для очищения списка коэффициентов от ведущих нулей

maxCoeff	number	-1.79769e+308 до 1.79769+308	Максимальный коэффициент полинома
bound	number	-1.79769e+308 до 1.79769+308	Граница диапазона поиска корней
step	number	-1.79769e+308 до 1.79769+308	Шаг, с которым будет искаться диапазон поиска корней
n	number	0 до 6	Степень полинома
k	number	0 до 7	Количество коэффициентов
newCoeffs	number[]	-	Новые коэффициенты полинома поле понижения степени
D	number	-1.79769e+308 до 1.79769+308	Дискриминант полинома 2 степени
a, b, c, d, n, m, k	number	-1.79769e+308 до 1.79769+308	Коэффициенты полинома при степени 6, 5, 4, 3, 2, 1, 0 соответственно
A, B, C	number	-1.79769e+308 до 1.79769+308	Вспомогательные переменные для нахождения коэффициентов полинома 3 степени
p, q	number	-1.79769e+308 до 1.79769+308	Коэффициенты полинома 3 степени
Q	number	-1.79769e+308 до 1.79769+308	Дискриминант полинома 3 степени
shift	number	-1.79769e+308 до 1.79769+308	Сдвиг для возврата к корням исходного полинома 3 степени
u, v	number	-1.79769e+308 до 1.79769+308	Вспомогательные переменные для нахождения корней полинома 3 степени
x1, x2	number	-1.79769e+308 до 1.79769+308	Корни полинома 3 степени
r	number	-1.79769e+308 до	Вспомогательная

		1.79769+308	переменная для нахождения угла ϕ
ϕ	number	-1.79769e+308 до 1.79769+308	Угол, необходимый для нахождения трёх корней полинома 3 степени
t	number	-1.79769e+308 до 1.79769+308	Вспомогательная переменная для нахождения корней полинома 3 степени
fx	number	-1.79769e+308 до 1.79769+308	Значение полинома в точке x
dfx	number	-1.79769e+308 до 1.79769+308	Значение производной полинома в точке x
delta	number	-1.79769e+308 до 1.79769+308	Шаг Ньютона для метода касательных
xPrev	number	-1.79769e+308 до 1.79769+308	Координата пересечения хорды с осью X в начале итерации
fl	number	-1.79769e+308 до 1.79769+308	Значение функции в точке l - левой границы
fr	number	-1.79769e+308 до 1.79769+308	Значение функции в точке r - правой границы

3. БЛОК СХЕМА АЛГОРИТМА

Блок схема алгоритма представлена в Приложении А.

4. ТЕСТИРОВАНИЕ АЛГОРИТМА

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:

> 1 -5 4 10 -11 -5 6

Введите по порядку через пробел:

eps - точность (1e-10)

N - количество итераций (1000)

x0 - начальное значение (0)

l - левая граница (-12)

r - правая граница (12)

Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите символ "-" на месте нужного параметра:

>

Найденные корни: -1 1 2 3

Рисунок 1 — Результат успешной работы алгоритма

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:

> 0 1 5 -6 -44 8 96

Введите по порядку через пробел:

eps - точность (1e-10)

N - количество итераций (1000)

x0 - начальное значение (0)

l - левая граница (-97)

r - правая граница (97)

Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите символ "-" на месте нужного параметра:

>

Найденные корни: -4 -3 -2 2

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:

> 0 0 1 1 -2 -3 -3

Введите по порядку через пробел:

eps - точность (1e-10)

N - количество итераций (1000)

x0 - начальное значение (0)

l - левая граница (-4)

r - правая граница (4)

Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите символ "-" на месте нужного параметра:

>

Найденные корни: -1.732 1.732

Рисунок 2 — Результат работы алгоритма для полиномов степени 5 и 4

```

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 0 0 0 1 -5 8 -4
Введите по порядку через пробел:
    eps - точность (1e-10)
    N - количество итераций (1000)
    x0 - начальное значение (0)
    l - левая граница (-9)
    r - правая граница (9)
Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите
символ "-" на месте нужного параметра:
>
Найденные корни: 1 2

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 0 0 0 0 1 -7 12
Введите по порядку через пробел:
    eps - точность (1e-10)
    N - количество итераций (1000)
    x0 - начальное значение (0)
    l - левая граница (-13)
    r - правая граница (13)
Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите
символ "-" на месте нужного параметра:
>
Найденные корни: 3 4

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 0 0 0 0 0 3 -6
Введите по порядку через пробел:
    eps - точность (1e-10)
    N - количество итераций (1000)
    x0 - начальное значение (0)
    l - левая граница (-3)
    r - правая граница (3)
Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите
символ "-" на месте нужного параметра:
>
Найденные корни: 2

```

Рисунок 3 — Результат работы алгоритма для полиномов степени 3, 2 и 1

```

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 1 2 3 4 5 6
Ошибка: Введено не 7 параметров. Повторите ввод:
> hello world 1 2 3 4 5
Ошибка: Не удалось прочитать числа или числа слишком большие. Повторите ввод:
> 1 2 3 4 5 6 1e1000
Ошибка: Не удалось прочитать числа или числа слишком большие. Повторите ввод:

```

Рисунок 4 — Результат работы алгоритма с ошибками при вводе коэффициентов

```

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 1 -5 4 10 -11 -5 6
Введите по порядку через пробел:
  eps - точность (1e-10)
  N - количество итераций (1000)
  x0 - начальное значение (0)
  l - левая граница (-12)
  r - правая граница (12)
Для установки всех значений по умолчанию нажмите "Enter". Для отдельного значения по умолчанию введите
символ "-" на месте нужного параметра:
> 1 2 3 4
Ошибка: Должно быть введено ровно 5 параметров. Повторите ввод:
> 1 2 3 4 d
Ошибка: Введены некорректные параметры. Повторите ввод:
> 1e-17 - - - -
Найденные корни: 2 3

```

Рисунок 5 — Результат работы алгоритма с ошибками при вводе параметров

```

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 0 0 0 0 0 0 0
Решением уравнения является любое число

```

Рисунок 6 — Результат работы алгоритма при всех нулевых коэффициентах

```

Введите 7 коэффициентов полинома (a b c d n m k) через пробел:
> 0 0 0 0 0 0 2
Решений нет

```

Рисунок 7 — Результат работы алгоритма при всех нулевых коэффициентах, кроме k

5. ИСХОДНЫЙ КОД АЛГОРИТМА

```
1 | const fs = require("fs");
2 |
3 | // -----
4 | //                               ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
5 | // -----
6 | /**
7 |  * Вычисляет значение полинома в точке x схемой Горнера
8 |  * @param {number[]} coeffs
9 |  * @param {number} x
10 |  */
11 | function f(coeffs, x) {
12 |   let res = 0;
13 |   for (let i = 0; i < coeffs.length; i++) res = res * x + coeffs[i];
14 |   return res;
15 | }
16 |
17 | /**
18 |  * Вычисляет значение производной функции в точке x схемой Горнера
19 |  * @param {number[]} coeffs
20 |  * @param {number} x
21 |  */
22 | function df(coeffs, x) {
23 |   let res = 0;
24 |   let n = coeffs.length - 1;
25 |   for (let i = 0; i < n; i++) res = res * x + coeffs[i] * (n - i);
26 |   return res;
27 | }
28 |
29 | /**
30 |  * Вычисляет значение второй производной функции в точке x схемой Горнера
31 |  * @param {number[]} coeffs
32 |  * @param {number} x
33 |  */
34 | function ddf(coeffs, x) {
35 |   let res = 0;
36 |   let n = coeffs.length - 1;
37 |   for (let i = 0; i < n - 1; i++)
38 |     res = res * x + coeffs[i] * (n - i) * (n - i - 1);
39 |   return res;
40 | }
41 |
42 | /**
43 |  * Вычисляет границы поиска корней по теореме Коши
44 |  * @param {number[]} coeffs
45 |  */
46 | function cacyBound(coeffs) {
47 |   let cleanedCoeffs = [];
48 |   let zeroFlag = false;
49 |   for (let i = 0; i < coeffs.length; i++) {
50 |     if (coeffs[i] !== 0 || zeroFlag) {
51 |       cleanedCoeffs.push(coeffs[i]);
52 |       zeroFlag = true;
53 |     }
54 |   }
55 |
56 |   if (cleanedCoeffs.length <= 1) return 100;
57 |
58 |   const maxCoeff =
59 |     cleanedCoeffs.length > 1
60 |       ? Math.max(...cleanedCoeffs.slice(1).map(Math.abs))
61 |       : 0;
62 |   return 1 + maxCoeff / Math.abs(cleanedCoeffs[0]);
63 | }
64 |
65 | /**
66 |  * Поиск интервала, где функция меняет свой знак
```

```

67 | * @param {number[]} coeffs
68 | * @param {number} bound
69 | */
70 | function findInterval(coeffs, bound) {
71 |     let step = bound / 20000;
72 |     let l = -bound;
73 |     while (l < bound) {
74 |         let r = l + step;
75 |         if (f(coeffs, l) * f(coeffs, r) < 0) return [l, r];
76 |         l += step;
77 |     }
78 |     return [-bound / 2, bound / 2];
79 | }
80 |
81 | // -----
82 | //                               МЕТОДЫ ДЛЯ НАХОЖДЕНИЯ КОРНЕЙ
83 | // -----
84 |
85 | /**
86 |  * Схема Горнера для понижения степени полинома.
87 |  * После деления на (x - root) уменьшает длину coeffs на единицу
88 |  * @param {number[]} coeffs
89 |  * @param {number} root
90 |  */
91 | function hornersShema(coeffs, root) {
92 |     const k = coeffs.length;
93 |     let newCoeffs = new Array(k - 1);
94 |     newCoeffs[0] = coeffs[0];
95 |     for (let i = 1; i < k - 1; i++) {
96 |         newCoeffs[i] = coeffs[i] + newCoeffs[i - 1] * root;
97 |     }
98 |     return newCoeffs;
99 | }
100 |
101 | /**
102 |  * Теорема Виета для полинома степени 2
103 |  * @param {number[]} coeffs
104 |  */
105 | function vietasTheorem(coeffs) {
106 |     const [a, b, c] = coeffs;
107 |     const D = b * b - 4 * a * c;
108 |     if (D >= 0)
109 |         return [(-b + Math.sqrt(D)) / (2 * a), (-b - Math.sqrt(D)) / (2 * a)];
110 |     if (D === 0) return [-b / (2 * a)];
111 |     else return null;
112 | }
113 |
114 | /**
115 |  * Метод Кардано для полинома степени 3
116 |  * @param {number[]} coeffs
117 |  */
118 | function cardanosMethod(coeffs) {
119 |     const [a, b, c, d] = coeffs;
120 |
121 |     let A = b / a;
122 |     let B = c / a;
123 |     let C = d / a;
124 |
125 |     const p = B - A ** 2 / 3;
126 |     const q = (2 * A ** 3) / 27 - (A * B) / 3 + C;
127 |     const Q = (p / 3) ** 3 + (q / 2) ** 2;
128 |     const shift = A / 3;
129 |
130 |     const cbrt = (x) => Math.sign(x) * Math.pow(Math.abs(x), 1 / 3);
131 |
132 |     if (Math.abs(Q) < 1e-5) {
133 |         const u = cbrt(-q / 2);
134 |         const x1 = 2 * u - shift;
135 |         const x2 = -u - shift;
136 |         return [x1, x2];
137 |     } else if (Q > 0) {

```

```

138     const u = cbrt(-q / 2 + Math.sqrt(Q));
139     const v = cbrt(-q / 2 - Math.sqrt(Q));
140     const x1 = u + v - shift;
141     return [x1];
142   } else {
143     const r = Math.sqrt(-(p ** 3) / 27);
144     const t = 2 * Math.sqrt(-p / 3);
145     const phi = (1 / 3) * Math.acos(-q / (2 * r));
146     return [
147       t * Math.cos(phi) - shift,
148       t * Math.cos(phi + (2 * Math.PI) / 3) - shift,
149       t * Math.cos(phi + (4 * Math.PI) / 3) - shift,
150     ];
151   }
152 }
153
154 /**
155  * Метод касательных
156  * @param {number[]} coeffs
157  * @param {number} eps - точность
158  * @param {number} N - количество итераций
159  * @param {number} x0 - предполагаемый корень
160  */
161 function tangentsMethod(coeffs, eps, N, x0) {
162   for (let i = 0; i < N; i++) {
163     const fx = f(coeffs, x0);
164     const dfx = df(coeffs, x0);
165     if (Math.abs(dfx) < 1e-14) {
166       if (Math.abs(fx) < eps) return x0;
167       else return null;
168     }
169     const delta = fx / dfx;
170     x0 -= delta;
171     if (Math.abs(delta) < eps) {
172       if (Math.abs(f(coeffs, x0)) < eps) return x0;
173     }
174   }
175   return null;
176 }
177
178 /**
179  * Метод хорд
180  * @param {number[]} coeffs
181  * @param {number} eps - точность
182  * @param {number} N - количество итераций
183  * @param {number} l - левая граница
184  * @param {number} r - правая граница
185  */
186 function chordsMethod(coeffs, eps, N, l, r) {
187   let x = l;
188   for (let i = 0; i < N; i++) {
189     let xPrev = x;
190     let fl = f(coeffs, l);
191     let fr = f(coeffs, r);
192     if (Math.abs(fr - fl) < 1e-12) {
193       if (Math.abs(f(coeffs, x)) < eps) return x;
194       l += eps;
195       continue;
196     }
197     x = l - (fl * (r - l)) / (fr - fl);
198     const fx = f(coeffs, x);
199     if (fx === 0) return x;
200     if (fl * fx < 0) r = x;
201     else l = x;
202     if (Math.abs(x - xPrev) < eps) {
203       if (Math.abs(fx) < eps) return x;
204     }
205   }
206   return null;
207 }
208

```



```

209  /**
210  * Комбинированный метод (метод хорд и касательных)
211  * @param {number[]} coeffs
212  * @param {number} eps - точность
213  * @param {number} N - количество итераций
214  * @param {number} l - левая граница
215  * @param {number} r - правая граница
216  */
217  function chordsAndTangentsMethod(coeffs, eps, N, l, r) {
218      for (let i = 0; i < N; i++) {
219          let fl = f(coeffs, l),
220              fr = f(coeffs, r);
221          if (Math.abs(fr - fl) < 1e-18) {
222              if (Math.abs(f(coeffs, (l + r) / 2)) < eps) return (l + r) / 2;
223              l += eps;
224              continue;
225          }
226          if (fr * ddf(coeffs, r) > 0) {
227              if (Math.abs(df(coeffs, r)) > 1e-15) r = r - fr / df(coeffs, r);
228              l = l - (fl * (r - l)) / (fr - fl);
229          } else {
230              if (Math.abs(df(coeffs, l)) > 1e-15) l = l - fl / df(coeffs, l);
231              r = r - (fr * (r - l)) / (fr - fl);
232          }
233          if (Math.abs(r - l) < eps) {
234              if (Math.abs(f(coeffs, (l + r) / 2)) < eps) return (l + r) / 2;
235          }
236      }
237      return null;
238  }
239
240  // -----
241  //             ФУНКЦИИ ДЛЯ ВАЛИДАЦИИ ПОЛЬЗОВАТЕЛЬСКОГО ВВОДА
242  // -----
243
244  /**
245  * Проверка числа
246  * @param {number} val
247  */
248  function isValidNumber(val) {
249      return isFinite(val) && Math.abs(val) <= Number.MAX_VALUE;
250  }
251
252  /**
253  * Запрос ввода строки
254  */
255  function readLine() {
256      const buffer = Buffer.alloc(500);
257      const bytesRead = fs.readSync(0, buffer, 0, 500);
258      if (bytesRead === 0) return "";
259      return buffer.toString("utf8", 0, bytesRead).trim();
260  }
261
262  /**
263  * Получение данных из пользовательского ввода
264  */
265  function getInput() {
266      let coeffsInput, eps, N, x0, l, r;
267      process.stdout.write(
268          "Введите 7 коэффициентов полинома (a b c d n m k) через пробел:\n> ",
269      );
270      while (true) {
271          let input = readLine();
272          if (!input) continue;
273          let parts = input.split(/\s+/);
274          if (parts.length !== 7) {
275              process.stdout.write(
276                  "Ошибка: Введено не 7 параметров. Повторите ввод:\n> ",
277              );
278              continue;
279          }

```

```

280 |     coeffsInput = parts.map(Number);
281 |     if (coeffsInput.some((t) => !isValidNumber(t))) {
282 |         process.stdout.write(
283 |             "Ошибка: Не удалось прочитать числа или числа слишком большие. Повторите ввод:
\n> ",
284 |             );
285 |         continue;
286 |     }
287 |     if (coeffsInput.every((t) => t === 0)) {
288 |         console.log("Решением уравнения является любое число");
289 |         process.exit(0);
290 |     }
291 |     if (
292 |         coeffsInput.slice(0, -1).every((c) => c === 0) &&
293 |         coeffsInput[6] !== 0
294 |     ) {
295 |         console.log("Решений нет");
296 |         process.exit(0);
297 |     }
298 |     break;
299 | }
300 |
301 | const bound = cachyBound(coeffsInput);
302 | let [lx, rx] = findInterval(coeffsInput, bound);
303 |
304 | let start;
305 | if (f(coeffsInput, lx) * ddf(coeffsInput, lx) > 0) start = lx;
306 | else start = rx;
307 |
308 | process.stdout.write(
309 |     "Введите по порядку через пробел:\n eps - точность (1e-10)\n N - количество
итераций (1000)\n x0 - начальное значение (0)\n l - левая граница (${bound})\n r - правая
граница (${bound})\nДля установки всех значений по умолчанию нажмите "Enter". Для отдельного
значения по умолчанию вводите символ "-" на месте нужного параметра:\n> `",
310 | );
311 | while (true) {
312 |     let input = readline();
313 |     const defaultValues = [1e-10, 1000, start, -bound, bound];
314 |     if (!input) {
315 |         [eps, N, x0, l, r] = defaultValues;
316 |         break;
317 |     }
318 |     let parts = input.split(/\s+/);
319 |     if (parts.length !== 5) {
320 |         process.stdout.write(
321 |             "Ошибка: Должно быть введено ровно 5 параметров. Повторите ввод:\n> ",
322 |             );
323 |         continue;
324 |     }
325 |     if (!parts.every((val) => isValidNumber(val) || val === "-")) {
326 |         process.stdout.write(
327 |             "Ошибка: Введены некорректные параметры. Повторите ввод:\n> ",
328 |             );
329 |         continue;
330 |     }
331 |
332 |     [eps, N, x0, l, r] = parts.map((val, ind) =>
333 |         val === "-" ? defaultValues[ind] : parseFloat(val),
334 |     );
335 |     break;
336 | }
337 |
338 | return [coeffsInput, eps, N, x0, l, r];
339 | }
340 |
341 | /**
342 |  * Вывод корней без повторов
343 |  * @param {number[]} solutions
344 |  */
345 | function printRoots(solutions) {
346 |     console.log(

```

```

347     "Найденные корни:",
348     ...new Set(solutions.map((val) => formatRoot(val)).sort((a, b) => a - b)),
349   );
350 }
351
352 /**
353  * Округление корня
354  * @param {number} x
355  */
356 function formatRoot(x) {
357   if (Math.abs(Math.round(x * 1e6) / 1e6 - Math.round(x)) < 0.1) {
358     return Math.round(x);
359   }
360   return parseFloat(x.toFixed(3));
361 }
362
363 // -----
364 //                               ГЛАВНАЯ ФУНКЦИЯ
365 // -----
366 function main() {
367   let solutions = [];
368   let [coeffs, eps, N, x0, l, r] = getInput();
369
370   let root;
371   // ПЕРВЫЙ КОРЕНЬ (6 степень)
372   if (coeffs[0] !== 0) {
373     root = tangentsMethod(coeffs, eps, N, x0);
374
375     if (root !== null) {
376       solutions.push(root);
377       coeffs = hornersShema(coeffs, root);
378     } else {
379       console.log("Корней нет");
380       return;
381     }
382   } else {
383     coeffs = coeffs.slice(1);
384   }
385
386   // ВТОРОЙ КОРЕНЬ (5 степень)
387   if (coeffs[0] !== 0) {
388     root = chordsMethod(coeffs, eps, N, ...findInterval(coeffs, r));
389     if (root !== null) {
390       solutions.push(root);
391       coeffs = hornersShema(coeffs, root);
392     } else {
393       printRoots(solutions);
394       return;
395     }
396   } else {
397     coeffs = coeffs.slice(1);
398   }
399
400   // ТРЕТИЙ КОРЕНЬ (4 степень)
401   if (coeffs[0] !== 0) {
402     root = chordsAndTangentsMethod(coeffs, eps, N, l, r);
403     if (root !== null) {
404       solutions.push(root);
405       coeffs = hornersShema(coeffs, root);
406       coeffs = coeffs.map((c) => (Math.abs(c) < 1e-12 ? 0 : c));
407     } else {
408       printRoots(solutions);
409       return;
410     }
411   } else {
412     coeffs = coeffs.slice(1);
413   }
414
415   let roots;
416   // МЕТОД КАРДАНО (3 степень)
417   if (coeffs[0] !== 0) {

```

```

418 |         roots = cardanosMethod(coeffs);
419 |         solutions.push(...roots);
420 |         printRoots(solutions);
421 |         return;
422 |     } else {
423 |         coeffs = coeffs.slice(1);
424 |     }
425 |
426 |     // ТЕОРЕМА ВЬЕТА (2 степень)
427 |     if (coeffs[0] !== 0) {
428 |         roots = vietasTheorem(coeffs);
429 |         solutions.push(...roots);
430 |         printRoots(solutions);
431 |         return;
432 |     } else {
433 |         coeffs = coeffs.slice(1);
434 |     }
435 |
436 |     // ЛИНЕЙНОЕ УРАВНЕНИЕ (1 степень)
437 |     if (coeffs[0] !== 0) {
438 |         solutions.push(-coeffs[1] / coeffs[0]);
439 |         printRoots(solutions);
440 |         return;
441 |     }
442 | }
443 |
444 | try {
445 |     main();
446 | } catch (e) {
447 |     if (e.message.includes("EOF")) console.log("\nВыход из программы.");
448 |     else console.error(e);
449 | }

```

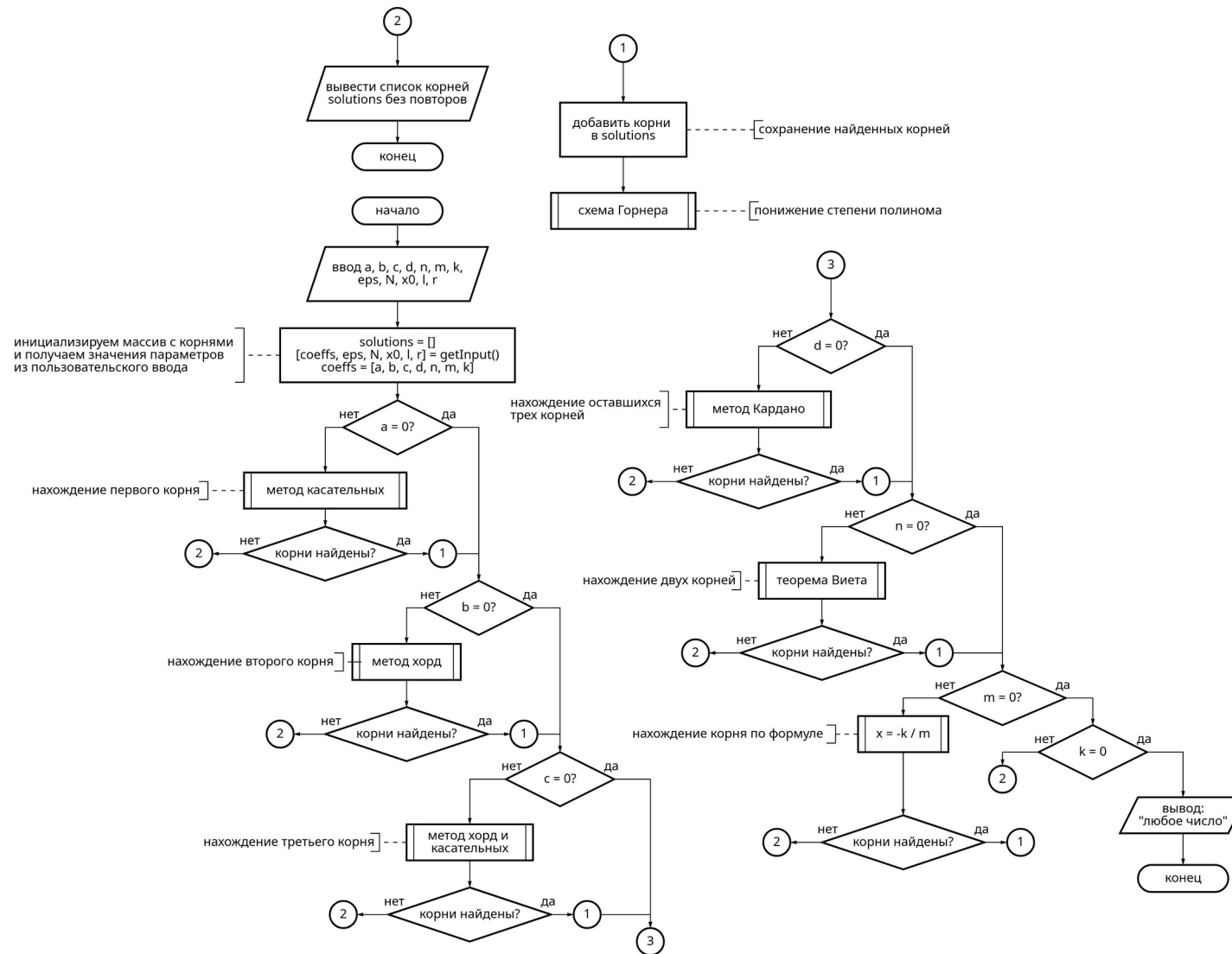
ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана программа на языке JavaScript для нахождения корней полинома шестой степени различными численными методами. Для написания исходного текста использовался редактор VSCodium, а запуск осуществлялся в среде выполнения NodeJS v25.0.7.

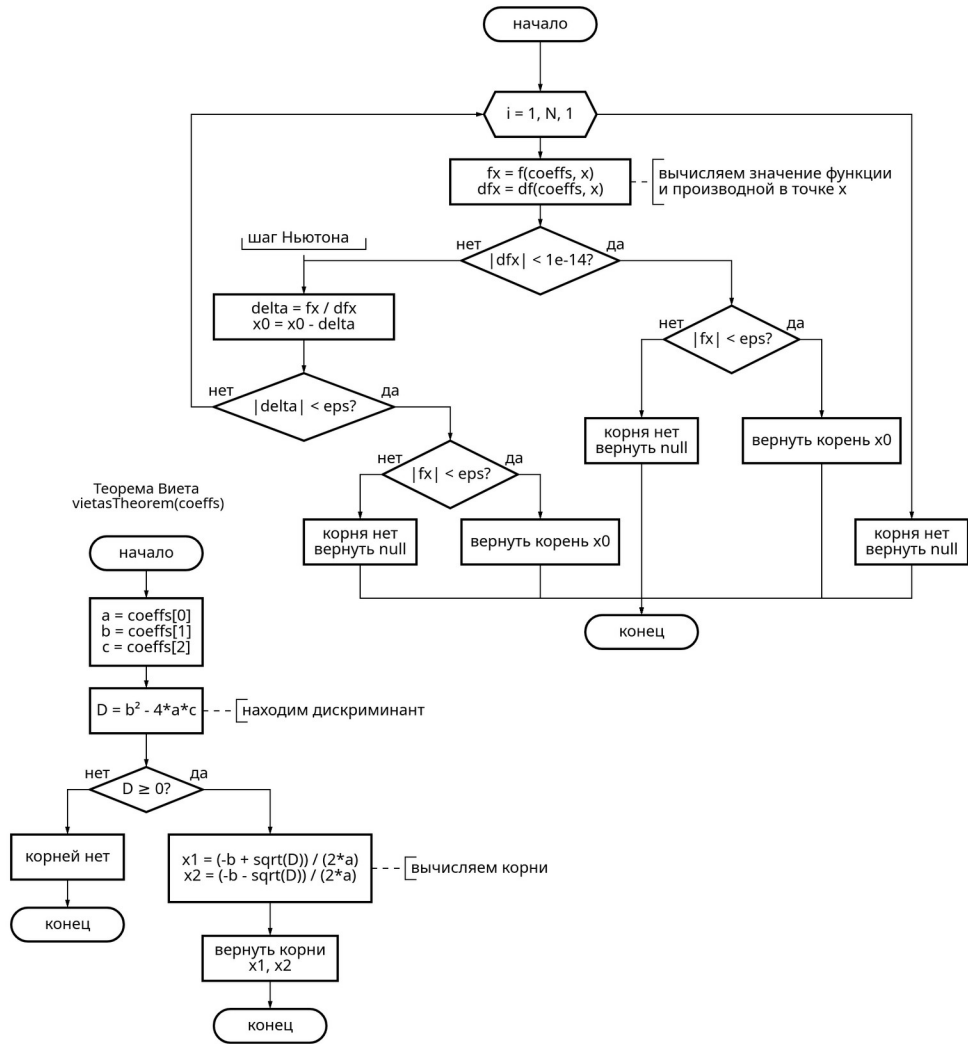
Были изучены методы нахождения корней: касательных, хорд, комбинированный хорд и касательных, а также метод Кардано, теорема Виета и схема Горнера (для понижения степени полинома).

Тестирование показало корректную работу программы при различных входных данных. Выполнение работы позволило закрепить навыки разработки алгоритмов для решения математических задач.

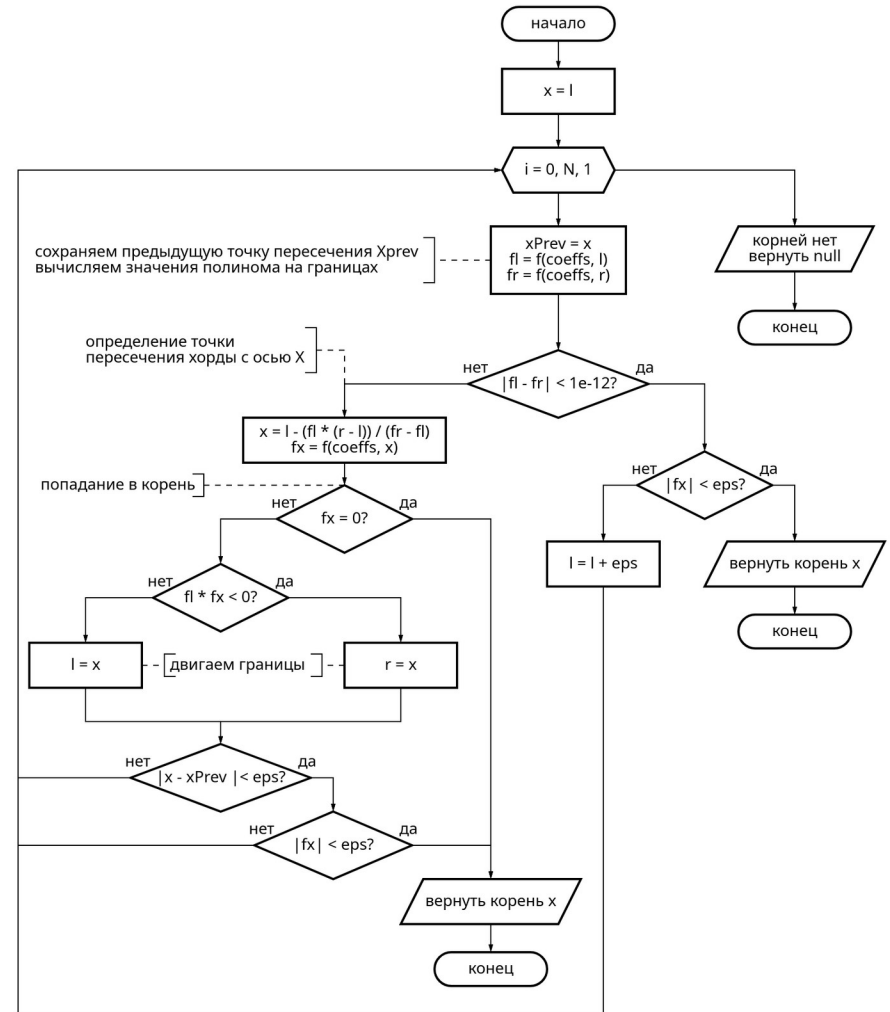
ПРИЛОЖЕНИЕ А

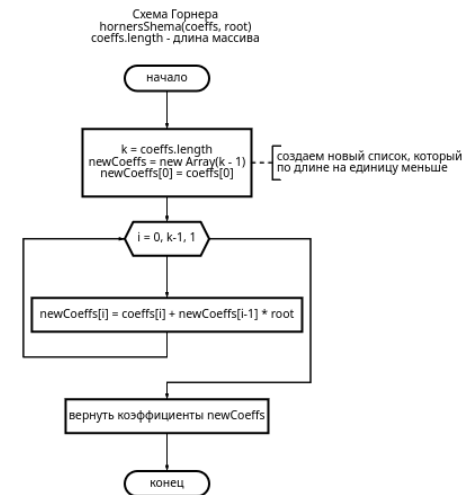
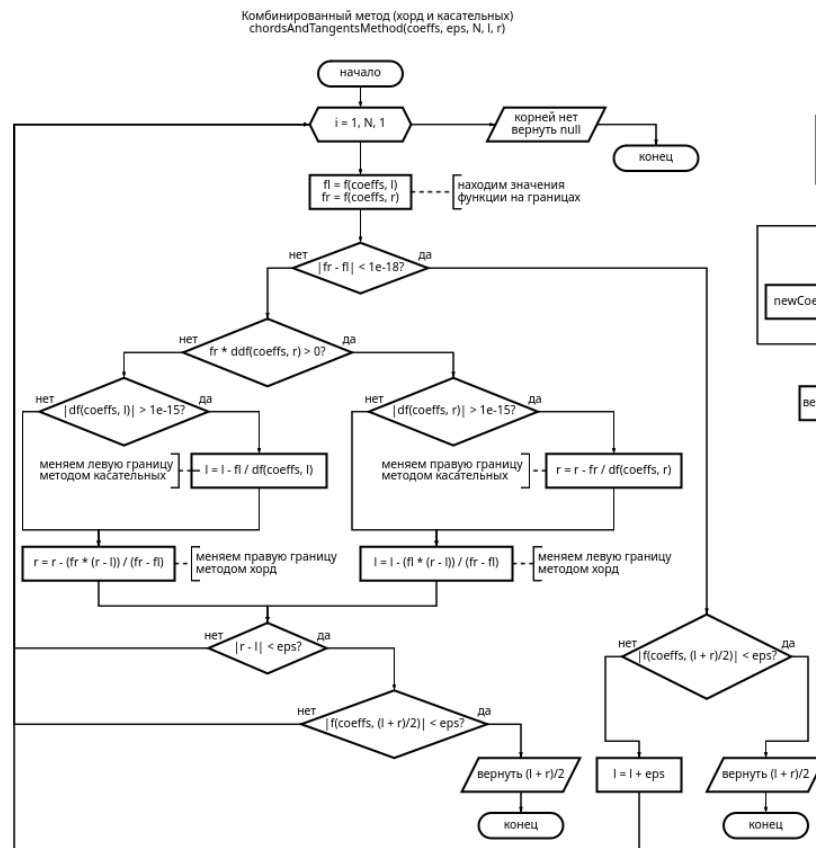
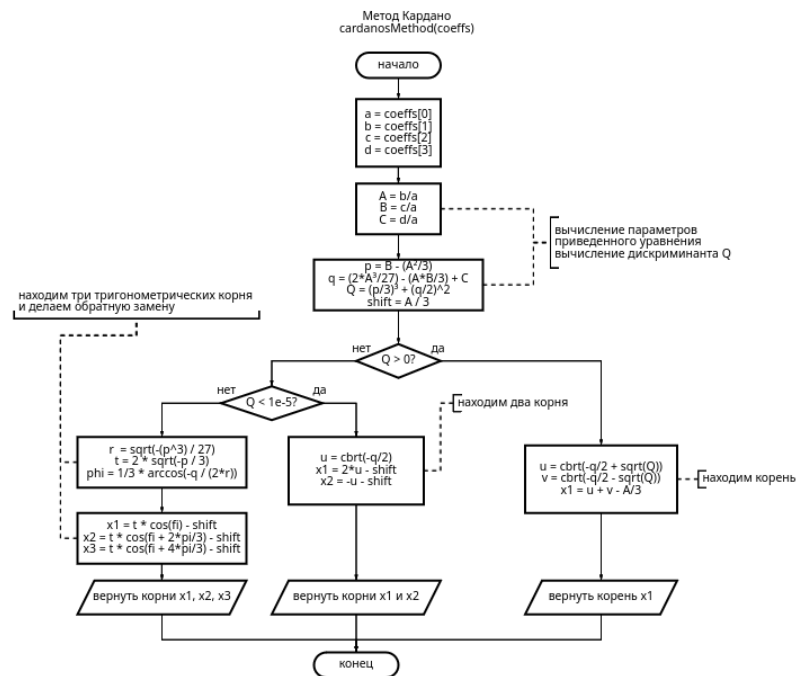


Метод касательных
tangentsMethod(coeffs, eps, N, x0)

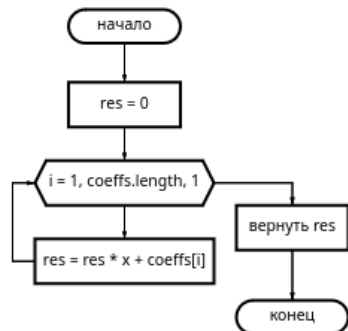


Метод хорд
chordsMethod(coeffs, eps, N, l, r)

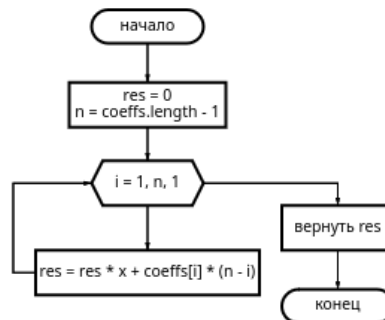




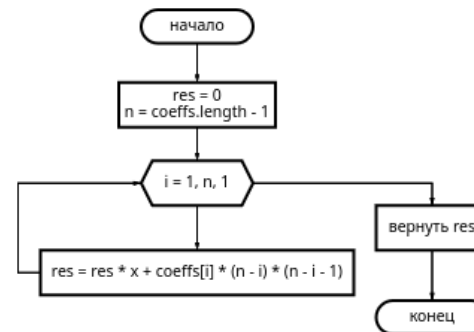
функция f
 $f(\text{coeffs}[\text{number}], x)$
 coeffs.length - длина массива



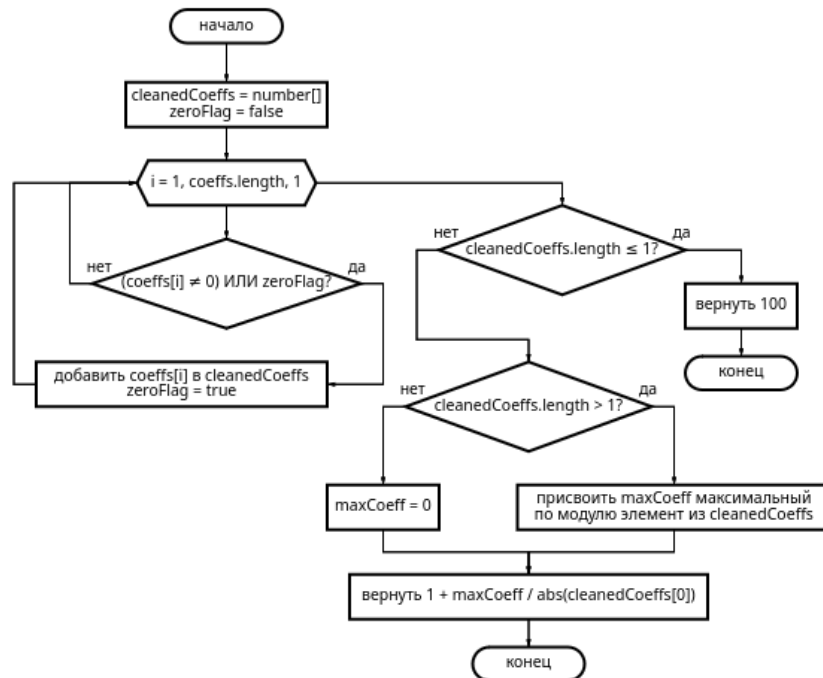
производная функции df
 $df(\text{coeffs}[\text{number}], x)$
 coeffs.length - длина массива



вторая производная функции ddf
 $ddf(\text{coeffs}[\text{number}], x)$
 coeffs.length - длина массива



теорема Коши для поиска границ корня
 $\text{casyBound}(\text{coeffs}[\text{number}])$
 coeffs.length - длина массива



поиск интервала со сменой знака функции
 $\text{findInterval}(\text{coeffs}[\text{number}], \text{bound})$
 coeffs.length - длина массива

